

Hands-On Data Analytics with Python

Eli Zamir

Session 1: Introduction

Self-introduction

Eli Zamir

E-Mail: Eli.Zamir@hs-aalen.de

Professional background:

Scientific research in life sciences with a strong focus on computational approaches

	Life-science domain	Computational aspects
B.Sc.	Biology	Interest in complex systems and their study
M.Sc.	Developmental biology	Tracking cells in embryos
Ph.D.	Cell biology	Image analysis
Postdoc I	Multiplexed optical methods	High-dimensional data analysis
Postdoc II	Systems biology	Spectroscopic data analysis, network analysis
Group leader	Systems biology	Analysis of high-dimensional, imaging and networks data
Project leader	Synthetic biology	Spectroscopic data analysis
Computational biology	Microbiome-cancer relations	Bioinformatics, big-data, high-dimensional data analysis

Course structure and schedule

Course structure

- Total sessions 12 - 13
- Each session consists of 2 parts: a teaching lecture and a hands-on part

Session dates and times

Times: Unless indicated otherwise, sessions take place from 14:30 to 17:45.

Dates:

Session 1: March 19

Session 2: March 26

Session 3: April 09

Session 4: April 16

Session 5: April 23

Session 6: April 30

Session 7: May 07

Session 8: May 21

Session 9: June 11

Session 10: June 18

Session 11: June 25

Session 12: Date and time to be decided

Session 13: To be decided (whether it will take place and when)

Course grading

Grade will be given based on a final project.

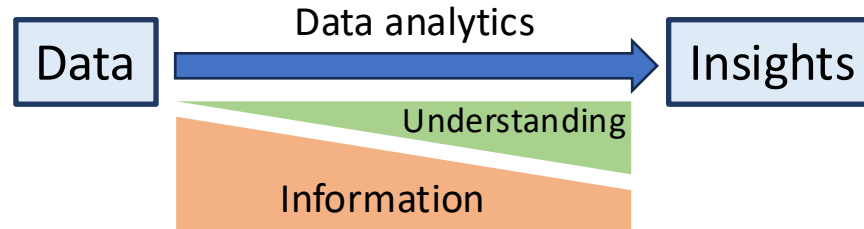
Work on the project should be done independently by each participant, outside of class hours.

The project will be given after Session 3.

The project will require:

- Getting an input data (will be provided)
- Applying the analysis modules that were learned
- Implementing also at least one analysis method (Python-based) which was not learned in the course
- Addressing given questions regarding the data
- Coming up with new questions and observations regarding the data, and assessing them as possible
- Submitting a final report in a form of a structured Jupyter-Notebook notebook, which should include:
Explanatory texts, code, figures and interpretation of the results

The objective of data analytics



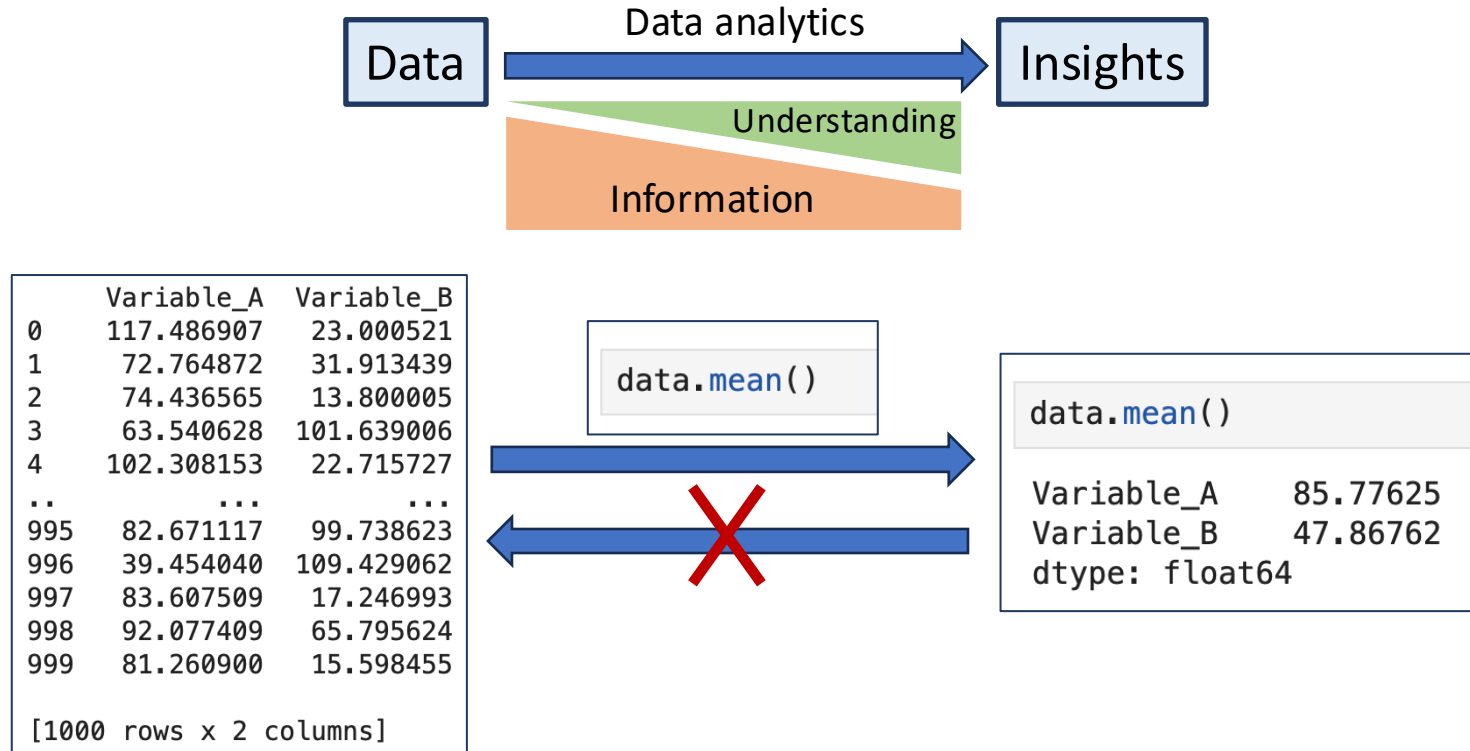
The goal of data analytics is deriving insights from the data

Insight: A discovered understanding

- Should be graspable
- Should be useful

From: Data analytics distills information in order to gain insights

Distilling information in order to gain insights – a simple example



Tabular data – a universal data structure

Raw data: The data in the form in which it got originally acquired

Most commonly, the data contains values for the **features** of **objects**

For example:

- Objects: Houses
- Features: area, location, price,... of each house

Such data has the form of a table (hence „tabular data“):

Data points (=“object”) X Features

- **Data point**: an individual observed example (“object”) within the dataset. (e.g., a given house)
- **Feature**: an individual property that describes each data point (e.g., price)

Tabular data – an example

Diagram illustrating the structure of tabular data:

- Features:** The columns of the table, representing the variables measured for each data point.
- Data points:** The rows of the table, representing individual observations (e.g., street blocks).

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422
...	...	...	...	...	...	...	...	...	...
20635	1.5603	25.0	5.045455	1.133333	845.0	2.560606	39.48	-121.09	0.781
20636	2.5568	18.0	6.114035	1.315789	356.0	3.122807	39.49	-121.21	0.771
20637	1.7000	17.0	5.205543	1.120092	1007.0	2.325635	39.43	-121.22	0.923
20638	1.8672	18.0	5.329513	1.171920	741.0	2.123209	39.43	-121.32	0.847
20639	2.3886	16.0	5.254717	1.162264	1387.0	2.616981	39.37	-121.24	0.894

Data points: street blocks

Features:

- MedInc: Median income
- HouseAge: Median age of houses
- AveRooms: Average number of rooms per house
- AveBedrms: Average number of bedrooms per house
- Population: Total population in the block group
- AveOccup: Average number of occupants
- Latitude: Geographic latitude
- Longitude: Geographic longitude
- MedHouseVal: Median house price

What could be useful insights to obtain from this data ?

Data

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422
...	...	...	...	...	...	...	...	...	...
20635	1.5603	25.0	5.045455	1.133333	845.0	2.560606	39.48	-121.09	0.781
20636	2.5568	18.0	6.114035	1.315789	356.0	3.122807	39.49	-121.21	0.771
20637	1.7000	17.0	5.205543	1.120092	1007.0	2.325635	39.43	-121.22	0.923
20638	1.8672	18.0	5.329513	1.171920	741.0	2.123209	39.43	-121.32	0.847
20639	2.3886	16.0	5.254717	1.162264	1387.0	2.616981	39.37	-121.24	0.894

data analytics



?

Insights

What could be useful insights to obtain from this data ?

Data

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422
...	...	...	...	...	...	...	...	...	...
20635	1.5603	25.0	5.045455	1.133333	845.0	2.560606	39.48	-121.09	0.781
20636	2.5568	18.0	6.114035	1.315789	356.0	3.122807	39.49	-121.21	0.771
20637	1.7000	17.0	5.205543	1.120092	1007.0	2.325635	39.43	-121.22	0.923
20638	1.8672	18.0	5.329513	1.171920	741.0	2.123209	39.43	-121.32	0.847
20639	2.3886	16.0	5.254717	1.162264	1387.0	2.616981	39.37	-121.24	0.894

data analytics



Insights

- Which features affect the most the price of a house?
- Can we predict the price of a house given its features?

But, why exactly is it difficult to "directly see" the insights?

Why exactly do we need computational approaches to derive insights from data?

- 1) Data sets are often too large, too complex, and too noisy for patterns to be directly apparent
- 2) Potential patterns must be quantified and statistically evaluated
- 3) Learned patterns are needed to be applied to new data to generate further insights and predictions

Data



- Which features affect the most the price of a house?
- Can we predict the price of a house given its features?
- Is the effect real or just random variation?
- How strong is the effect?
- ... based on learned patterns?

A combination of several factors makes it difficult to directly identify patterns in the data

(1) Many data points

- Integration of large amount of information is required
 - *E.g., data set with 27000 street blocks as data points*

(2) Multiple features

- Patterns can be multi-parametric (multi-dimensional) → harder to discover, grasp and visualize
 - *E.g., house price affected by multiple features (age, location, area,...) with interactions between them*

(3) Variance

- Obscures patterns, unless the features that causing the variance are also taken into account
 - *E.g., Price of houses with the same number of rooms can vary because of different house-ages. If house-age will not be considered, then the affect rooms on the price will be strongly obscured*

(4) Noise

- Obscures patterns
 - *E.g., price of houses with similar features can have fluctuations, due to measurement errors, decision noise, and other unknown affecting factors*

How data analytics overcomes data complexity to uncover patterns

Data-complexity aspect

(1) Many data points

- Integration of large amount of information is required

(2) Multiple features

- Patterns can be multi-parametric (multi-dimensional)

(3) Variance

- Obscures patterns, unless underline features are taken into account

(4) Noise

- Obscures patterns

Data analytics aspect



- ✓ Computationally easy
- ✓ Becomes highly advantageous for overcoming high-dimensionality, variance and noise



- ✓ Dimensionality reduction
- ✓ Identifying signatures
- ✓ Features extraction
- ✓ Features selection



- ✓ Takes into account multiple features
- ✓ Uses variance as a source of information



- ✓ Overcomes noise by integrating many data points
- ✓ Takes advantage of noise to reveal associations

The structure of the data is universal and so is the type of questions

Street block features

street
blocks

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.181467	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...
20635	1.5603	25.0	5.045455	1.133333	845.0	2.560606	39.48	-121.09	0.781
20636	2.5568	18.0	6.114035	1.315789	356.0	3.122807	39.49	-121.21	0.771
20637	1.7000	17.0	5.205543	1.120092	1007.0	2.325635	39.43	-121.22	0.923
20638	1.8672	18.0	5.329513	1.171920	741.0	2.123209	39.43	-121.32	0.847
20639	2.3886	16.0	5.254717	1.162264	1387.0	2.616981	39.37	-121.24	0.894

Questions:

Which house features affect the most its price?

Can we predict the price of a house given its features?

Proteins (levels)

Cells

	A	B	C	D	E	F	D	E ...
0								
1								
2								
3								
4								
...								
20635								
20636								
20637								
20638								
20639								

Questions:

Which proteins affect the most the cell behaviour ?

Can we predict the behaviour of a cell based on its protein levels ?

Learning goals

(1) Learning Python:

- As a programming language (core programming)
- As an interactive environment for exploring and working with data
- As an ecosystem for data analytics (libraries: Pandas, NumPy, SciPy, Matplotlib, Seaborn, Scikit-learn)

(2) Learning Jupyter Notebook:

- As an environment for data analysis, documentation and visualization

(3) Learning to handle and process data:

- Focusing on tabular data: data points $\times$ features

(4) Learning to perform statistical tests:

- Univariate tests
- Association tests

(5) Learning to visualize data and analysis results

(6) Learning to explore high-dimensional data:

- Cluster analyses
- Dimensionality reduction methods

(7) Learning to perform regression analyses:

- Predicting numerical outcomes using linear regression and polynomial regression
- Predicting binary categorical outcomes using logistic regression
- Assessing feature contributions to the outcome
- Regularization methods

(8) Learning to apply machine learning algorithms:

- Focusing on Random Forests
- Implementations for predicting numerical outcomes (regression) and categorical outcomes (classification)

Session topics

Session 1: Introduction

Session 2: Core Python programming (part 1)

Session 3: Core Python programming (part 2)

Session 4: Handling and processing data using the Pandas library

Session 5: Data visualization using the Matplotlib and Seaborn libraries

Session 6: Statistical Testing with SciPy

Session 7: Correlation analyses

Session 8: Processing of high-dimensional data; Cluster analyses

Session 9: Dimensionality reduction methods

Session 10: Regression analyses

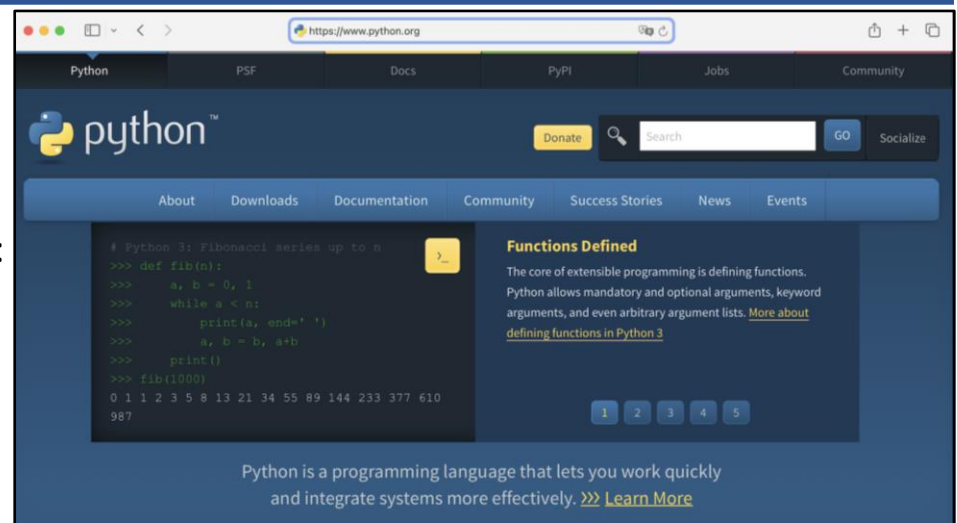
Session 11: Logistic regression

Session 12: Random forests

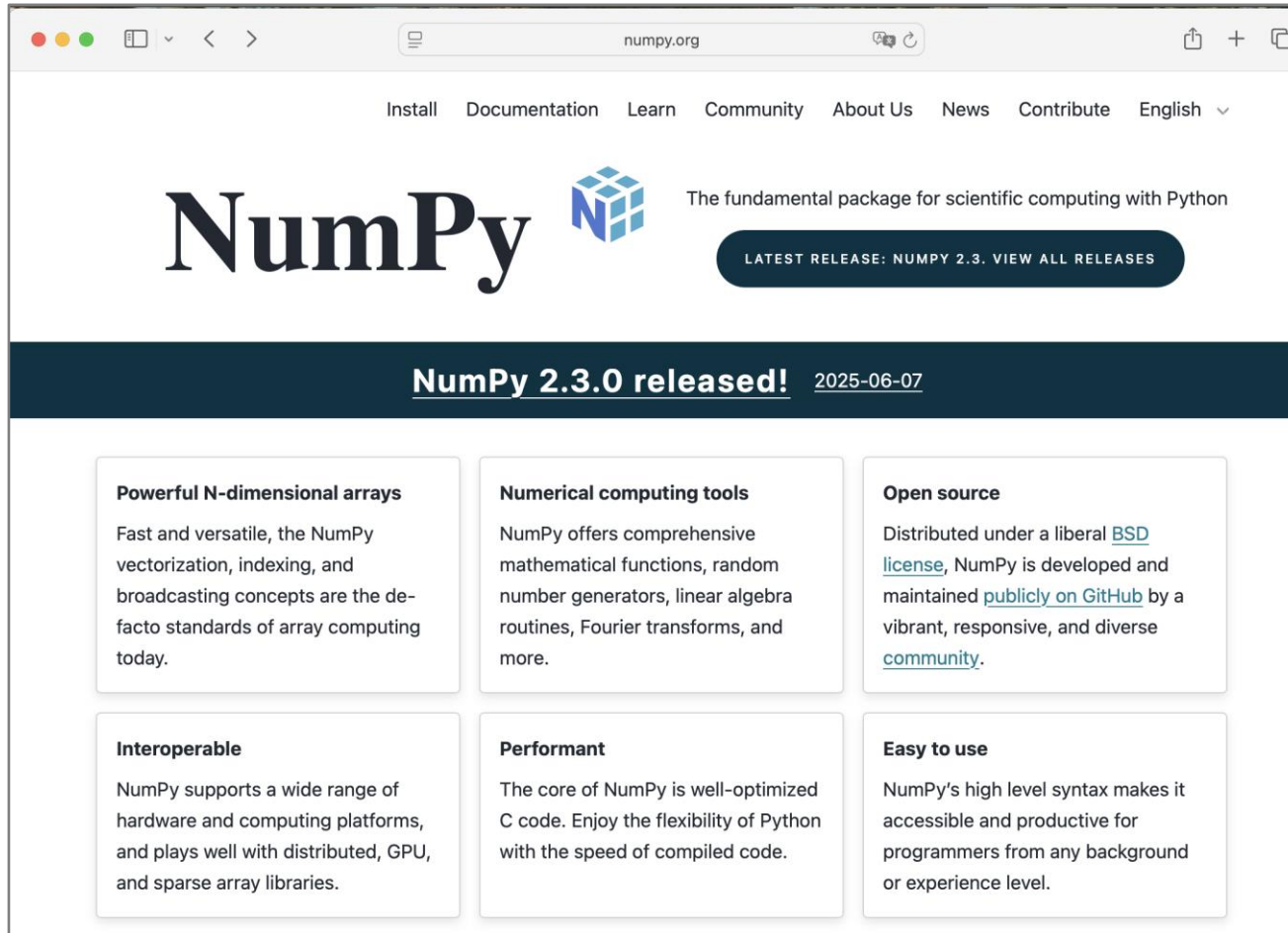
Optional session 13: To be decided on

Python main features

- General-purpose programming language
- Interpreted language (no need to compile)
→ easy to interact with the code, the data and the results
- Supports multiple programming paradigms:
 - Procedural programming
 - Object-oriented programming
 - Functional programming
- Large ecosystem of third-party:
 - NumPy (numerical computing)
 - pandas (data handling)
 - Matplotlib and Seaborn (data visualization)
 - SciPy (scientific computing and statistics)
 - scikit-learn (machine learning)
 - TensorFlow/PyTorch (deep learning)
- Large and active community of developers and users
- Cross-platform compatibility: Windows, macOS, Linux
- Open source and free
- High applicability across many domains
- Strong tools (Jupyter Notebook , IDEs, terminal)



“NumPy”: Library for scientific computing



The screenshot shows the NumPy website homepage in a web browser. The browser's address bar displays 'numpy.org'. The website has a navigation bar with links: 'Install', 'Documentation', 'Learn', 'Community', 'About Us', 'News', 'Contribute', and 'English'. The main header features the 'NumPy' logo, a 3D cube icon, and the tagline 'The fundamental package for scientific computing with Python'. A dark blue button highlights the 'LATEST RELEASE: NUMPY 2.3. VIEW ALL RELEASES'. A dark blue banner below the header announces 'NumPy 2.3.0 released!' with the date '2025-06-07'. The main content area consists of six white boxes with dark blue headers and text descriptions of NumPy's features.

Install Documentation Learn Community About Us News Contribute English

NumPy

The fundamental package for scientific computing with Python

LATEST RELEASE: NUMPY 2.3. VIEW ALL RELEASES

NumPy 2.3.0 released! 2025-06-07

Powerful N-dimensional arrays

Fast and versatile, the NumPy vectorization, indexing, and broadcasting concepts are the de-facto standards of array computing today.

Numerical computing tools

NumPy offers comprehensive mathematical functions, random number generators, linear algebra routines, Fourier transforms, and more.

Open source

Distributed under a liberal [BSD license](#), NumPy is developed and maintained [publicly on GitHub](#) by a vibrant, responsive, and diverse [community](#).

Interoperable

NumPy supports a wide range of hardware and computing platforms, and plays well with distributed, GPU, and sparse array libraries.

Performant

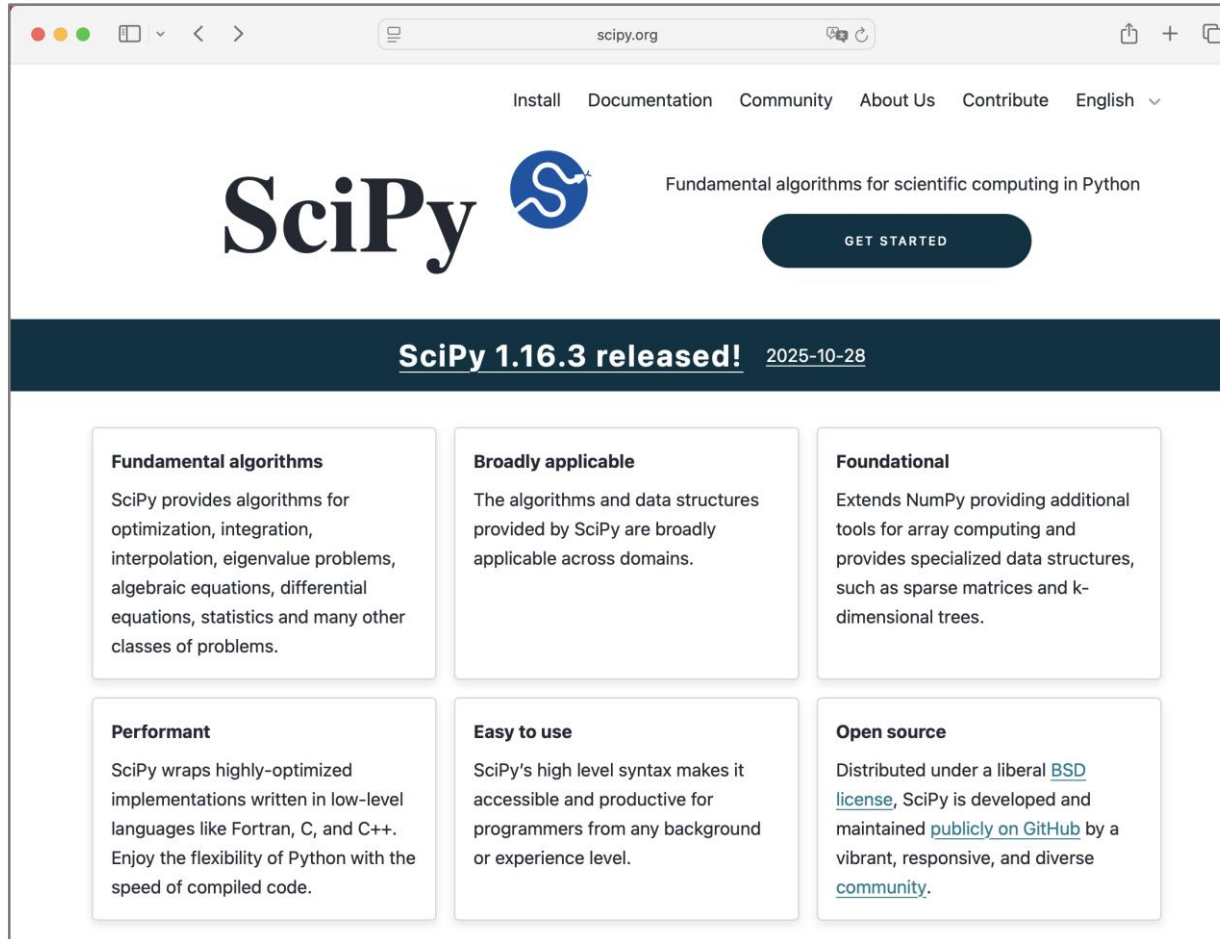
The core of NumPy is well-optimized C code. Enjoy the flexibility of Python with the speed of compiled code.

Easy to use

NumPy's high level syntax makes it accessible and productive for programmers from any background or experience level.

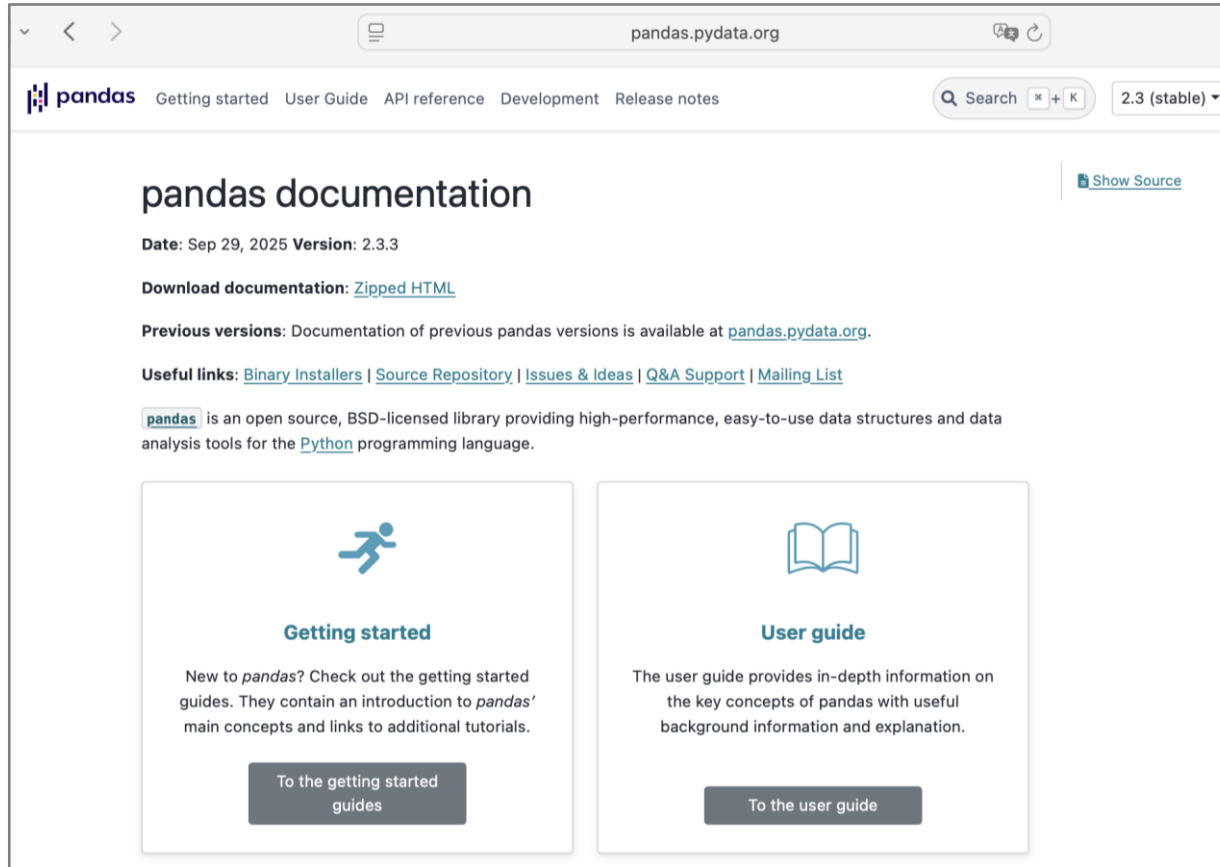
<https://numpy.org>

“SciPy”: Library for scientific computing and statistics



<https://scipy.org>

“pandas”: Library for handling and processing data



<https://pandas.pydata.org/docs/>

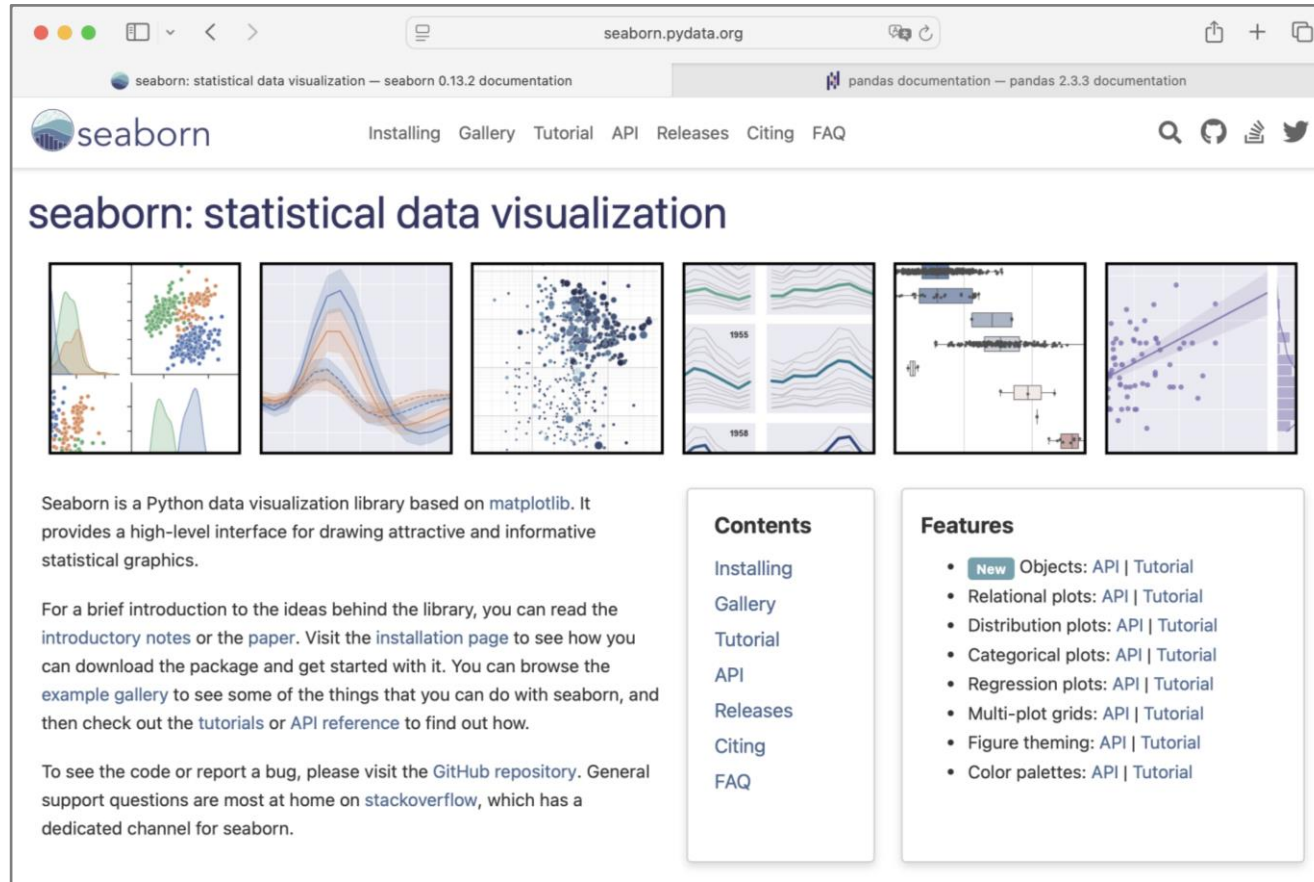
“Matplotlib”: Library for data visualization

The screenshot shows the Matplotlib website interface. At the top, there's a navigation bar with links: Plot types, User guide, Tutorials, Examples, Reference, Contribute, and Releases. The current version is 3.10 (stable). The main content area is titled 'Lines, bars and markers' under the 'Examples' section. On the left, there's a 'Section Navigation' sidebar with a list of topics: Infinite lines, Bar chart with individual bar colors, Bar chart with labels, Stacked bar chart, Grouped bar chart with labels, Horizontal bar chart, Broken horizontal bars, CapStyle, Plotting categorical variables, and Plotting the coherence of two signals. The main content area displays eight example plots in a grid:

- Infinite lines:** A plot showing a blue curve representing the function $y = \frac{1}{1 + e^{-x}}$ over the range $x \in [-10, 10]$.
- Bar chart with individual bar colors:** A bar chart titled 'Profit supply by kind and color' showing profit for different colors (red, blue, orange) across categories like 'apple', 'banana', 'cherry', and 'orange'.
- Bar chart with labels:** A stacked bar chart titled 'Number of penguins by sex' showing the count of penguins for different species (Adelie, Chinstrap, Gentoo) categorized by sex (Male, Female).
- Stacked bar chart:** A stacked bar chart titled 'Number of penguins with above average body mass' showing the count of penguins for different species categorized by sex.
- Horizontal bar chart:** A horizontal bar chart titled 'Penguin attributes by species' showing the length of penguins for different species (Adelie, Chinstrap, Gentoo) categorized by sex.
- How fast do you want to go today?:** A horizontal bar chart showing the number of times different people (A, B, C, D, E) have used different modes of transport (Bike, Car, Train, Bus, Foot).
- Resource usage:** A horizontal bar chart showing the number of times different people (A, B, C, D, E) have used different resources (CPU, Memory, Disk, Network).
- Cap style:** A plot showing three different bar styles: 'butt', 'round', and 'projecting'.

<https://matplotlib.org>

“Seaborn”: Library for statistical data visualization



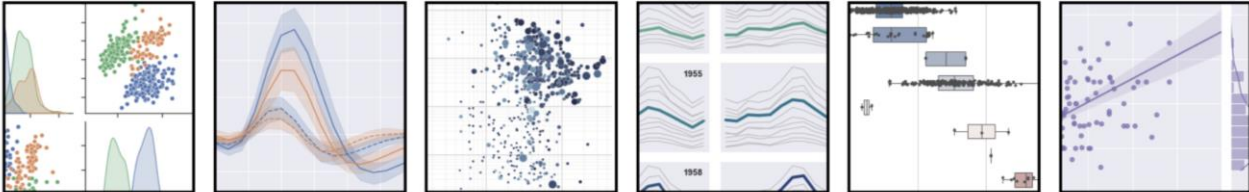
The screenshot shows the Seaborn website (seaborn.pydata.org) with a navigation bar including links for Installing, Gallery, Tutorial, API, Releases, Citing, and FAQ. The main heading is "seaborn: statistical data visualization". Below this is a row of six thumbnail images showing different types of plots: a faceted distribution plot, a density plot, a scatter plot, a faceted line plot, a box plot, and a regression plot. The text describes Seaborn as a Python data visualization library based on matplotlib, providing a high-level interface for drawing attractive and informative statistical graphics. It includes links to introductory notes, the paper, installation page, example gallery, tutorials, and API reference. It also mentions where to find code, report bugs, or get support. On the right, there are two sections: "Contents" with links to Installing, Gallery, Tutorial, API, Releases, Citing, and FAQ; and "Features" with a list of capabilities including Objects, Relational plots, Distribution plots, Categorical plots, Regression plots, Multi-plot grids, Figure theming, and Color palettes.

seaborn: statistical data visualization — seaborn 0.13.2 documentation

seaborn

Installing Gallery Tutorial API Releases Citing FAQ

seaborn: statistical data visualization



Seaborn is a Python data visualization library based on `matplotlib`. It provides a high-level interface for drawing attractive and informative statistical graphics.

For a brief introduction to the ideas behind the library, you can read the [introductory notes](#) or the [paper](#). Visit the [installation page](#) to see how you can download the package and get started with it. You can browse the [example gallery](#) to see some of the things that you can do with seaborn, and then check out the [tutorials](#) or [API reference](#) to find out how.

To see the code or report a bug, please visit the [GitHub repository](#). General support questions are most at home on [stackoverflow](#), which has a dedicated channel for seaborn.

Contents

- [Installing](#)
- [Gallery](#)
- [Tutorial](#)
- [API](#)
- [Releases](#)
- [Citing](#)
- [FAQ](#)

Features

- New** Objects: [API](#) | [Tutorial](#)
- Relational plots: [API](#) | [Tutorial](#)
- Distribution plots: [API](#) | [Tutorial](#)
- Categorical plots: [API](#) | [Tutorial](#)
- Regression plots: [API](#) | [Tutorial](#)
- Multi-plot grids: [API](#) | [Tutorial](#)
- Figure theming: [API](#) | [Tutorial](#)
- Color palettes: [API](#) | [Tutorial](#)

<https://seaborn.pydata.org>

“Scikit-learn”: Library for machine learning

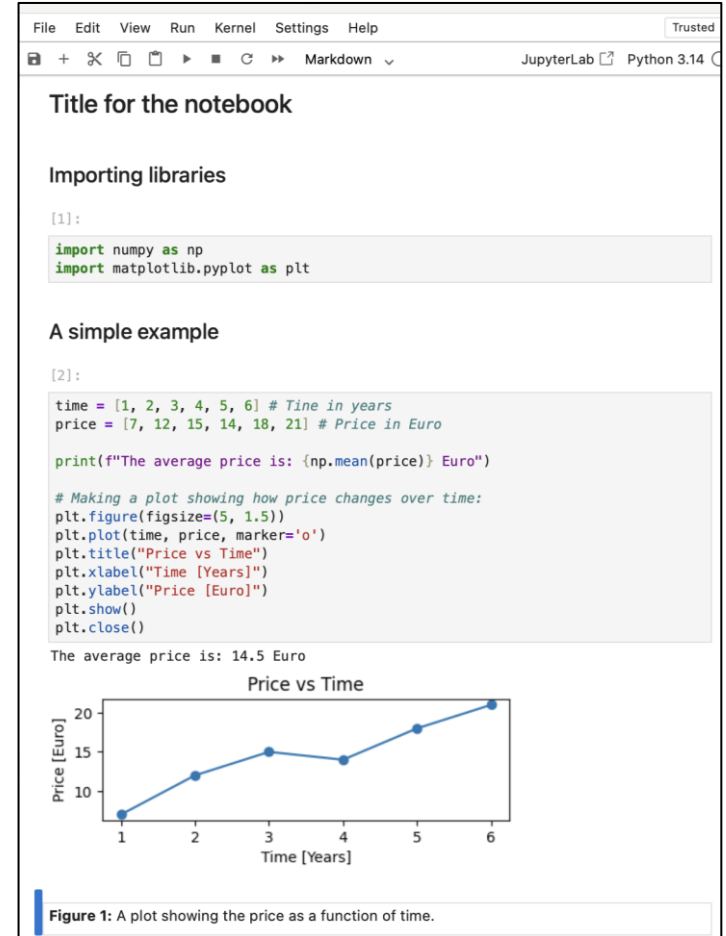
The screenshot shows the scikit-learn.org homepage. At the top, there's a navigation bar with links for 'Install', 'User Guide', 'API', 'Examples', 'Community', and 'More'. The main header features the 'scikit-learn' logo and the tagline 'Machine Learning in Python'. Below this, there are buttons for 'Getting Started' and 'Release Highlights for 1.7'. A list of key features is displayed: 'Simple and efficient tools for predictive data analysis', 'Accessible to everybody, and reusable in various contexts', 'Built on NumPy, SciPy, and matplotlib', and 'Open source, commercially usable - BSD license'. The page is organized into six categories, each with a title, description, applications, algorithms, and a visual example:

- Classification**: Identifying which category an object belongs to. Applications: Spam detection, image recognition. Algorithms: Gradient boosting, nearest neighbors, random forest, logistic regression, and more... Example: A 4x4 grid of 16 small plots showing various classification results on different datasets.
- Regression**: Predicting a continuous-valued attribute associated with an object. Applications: Drug response, stock prices. Algorithms: Gradient boosting, nearest neighbors, random forest, ridge, and more... Example: A line plot titled 'Predicted average energy transfer during the week' showing recorded average, max, and min values over the days of the week.
- Clustering**: Automatic grouping of similar objects into sets. Applications: Customer segmentation, grouping experiment outcomes. Algorithms: k-Means, HDBSCAN, hierarchical clustering, and more... Example: A scatter plot titled 'Kmeans clustering on the digits dataset (PCA-reduced data)' showing data points grouped into clusters with centroids marked by white crosses.
- Dimensionality reduction**: Reducing the number of random variables to consider. Applications: Visualization, increased efficiency. Algorithms: PCA, feature selection, non-negative matrix factorization, and more... Example: (Not explicitly shown in the image, but the category is present).
- Model selection**: Comparing, validating and choosing parameters and models. Applications: Improved accuracy via parameter tuning. Algorithms: Grid search, cross validation, metrics, and more... Example: (Not explicitly shown in the image, but the category is present).
- Preprocessing**: Feature extraction and normalization. Applications: Transforming input data such as text for use with machine learning algorithms. Algorithms: Preprocessing, feature extraction, and more... Example: (Not explicitly shown in the image, but the category is present).

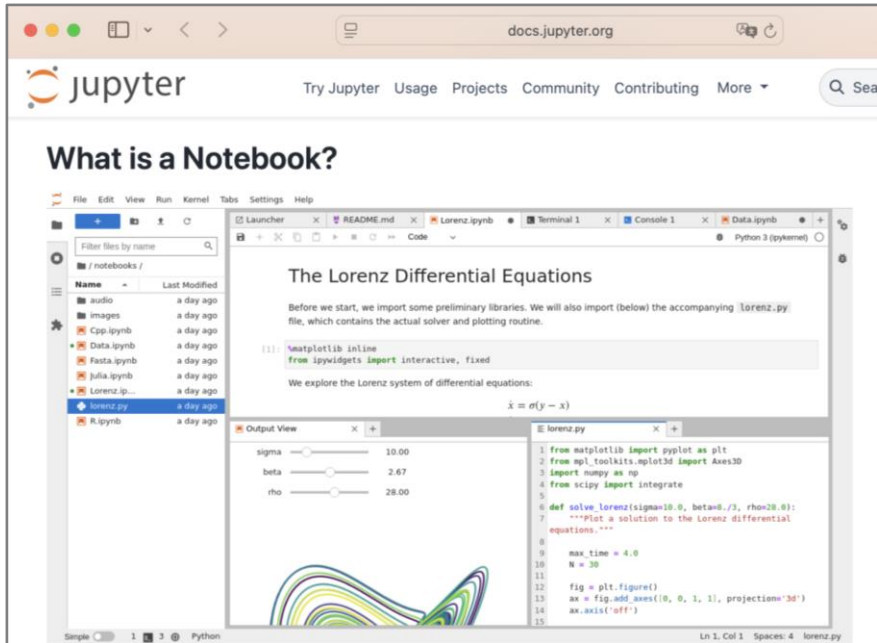
<https://scikit-learn.org/>

Jupyter Notebook

- The notebook combines in one interactive document code, visualizations and explanatory text
- The notebook consists of cells: code cells and markdown cells
- The kernel executes the code in the notebook and keeps track of the variables and the imported libraries
- In addition to Python, Jupyter Notebook also supports other languages, such as R



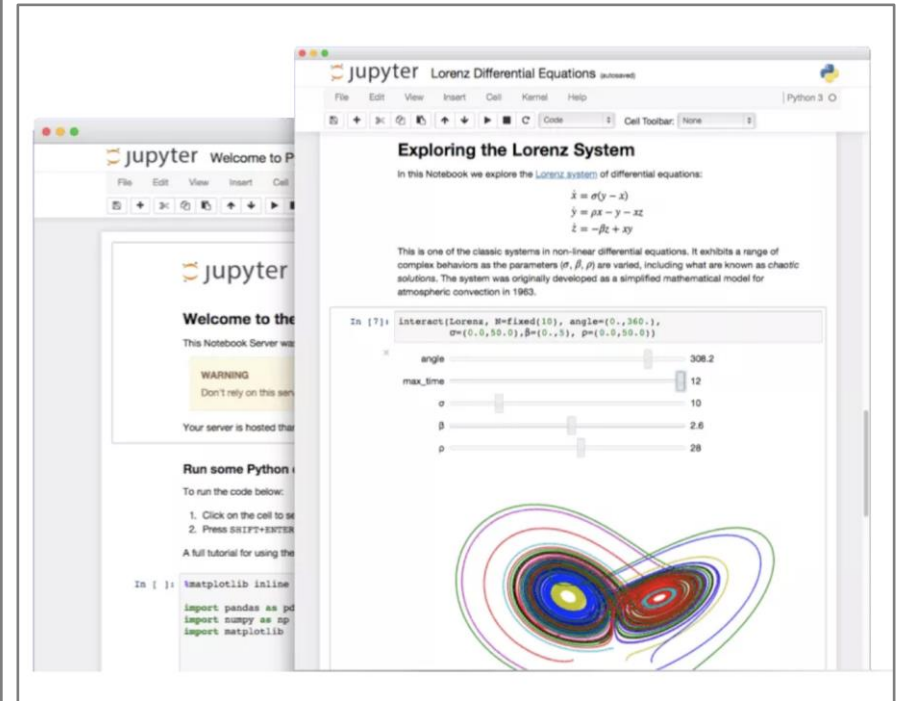
Jupyter Lab / Jupyter Notebook



Pictured: A computational notebook document, shown inside JupyterLab

Note: Read [What is Jupyter?](#) for a detailed look at Jupyter and notebooks.

A notebook is a shareable document that combines computer code, plain language descriptions, data, rich visualizations like 3D models, charts, graphs and figures, and interactive controls. A notebook, along with an editor (like JupyterLab), provides a fast interactive environment for prototyping and explaining code, exploring and visualizing data, and sharing ideas with others.



<https://docs.jupyter.org/>

Course: Hands-On Data Analytics with Python

Eli Zamir

Session 1: Introduction - Hands-on session

Session 1: Hand-on Tasks

- (1) Installing Python
- (2) Installing Jupyter Notebook
- (3) Installing 6 Python libraries: Pandas, Numpy, SciPy, Matplotlib, Seaborn, scikit-learn
- (4) Learning and practicing key Jupyter Notebook features

(1) Installing Python

Step 1. Go to the official Python website:

<https://www.python.org/downloads>

The website should automatically detect your operating system

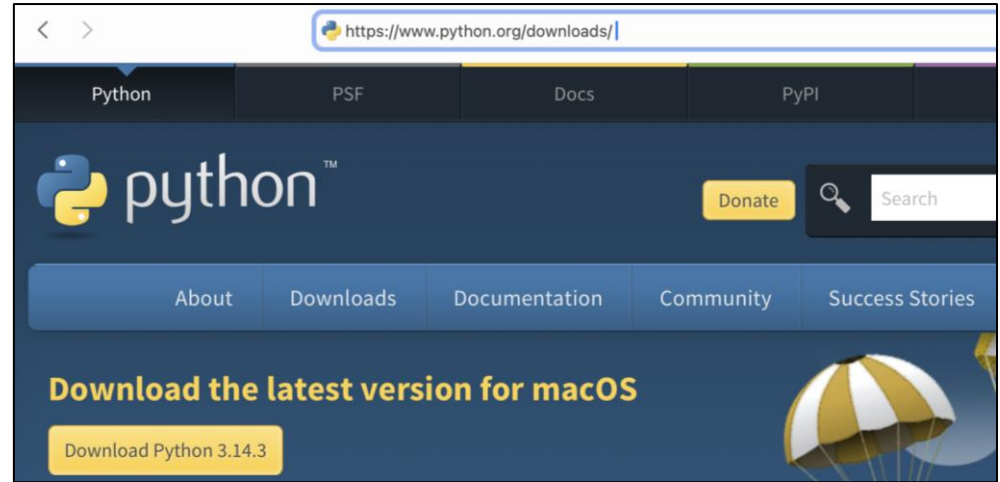
Step 2. Click on “Download Python 3.14.3”

Step 3. Run the installer:

- For Windows operating system:
 - a) Open the downloaded .exe installer
 - b) Check the box "Add Python to PATH" **before** clicking Install Now (!)
 - c) Follow the prompts to complete the installation.
- For macOS:
 - a) Open the downloaded .pkg installer file
 - b) Follow the installer instructions to install Python

Step 4. Verify the installation:

- a) Open Terminal (macOS) or Command Prompt / PowerShell (Windows)
- b) Type and execute the command: `python3 --version`
 - The output should be: “Python 3.14.3”
- c) Type and execute the command: `pip3 --version`
 - The output should have the pattern: “pip From ...”



(2) Installing Jupyter Notebook

Step 1: Open Terminal (macOS) or Command Prompt / PowerShell (Windows)

Step 2: Type and execute the command: `python3 -m pip install notebook`

[notice] A new release of pip is available: 25.3 -> 26.0.1

[notice] To update, run: `pip3 install --upgrade pip`

```
python3 -m pip install --upgrade pip
```

(3) Installing Python libraries

Step 1: Open Terminal (macOS) or Command Prompt / PowerShell (Windows)

Step 2: Type and execute one-by-one the following 6 commands, to install the 6 libraries:

- `python3 -m pip install -U pandas`
- `python3 -m pip install -U numpy`
- `python3 -m pip install -U scipy`
- `python3 -m pip install -U matplotlib`
- `python3 -m pip install -U seaborn`
- `python3 -m pip install -U scikit-learn`

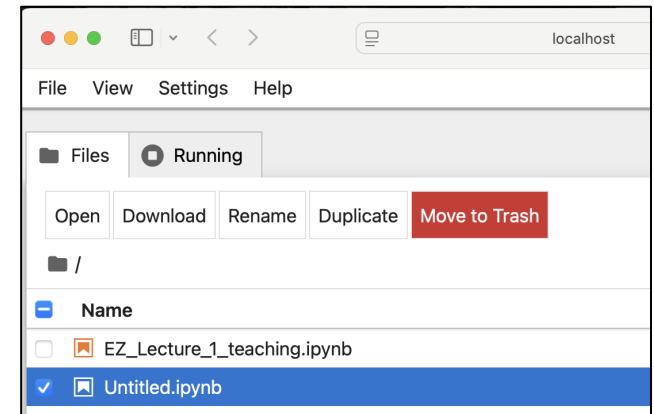
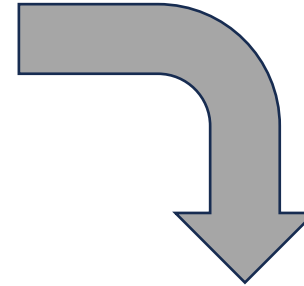
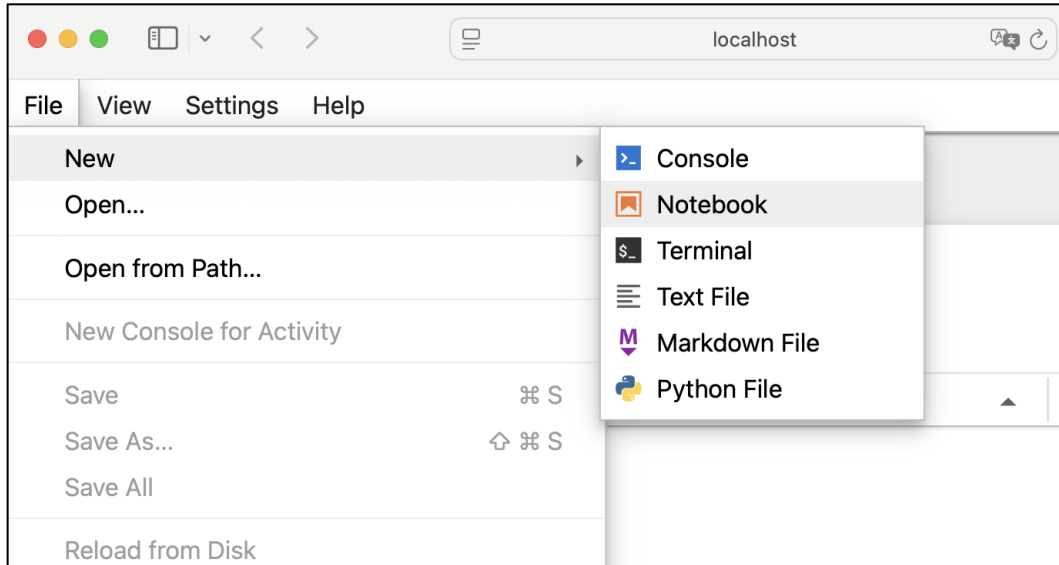
Launching Jupyter Notebook

Step 1: Open Terminal (macOS) or Command Prompt / PowerShell (Windows)

Step 2: Type and execute the command: `python3 -m notebook`

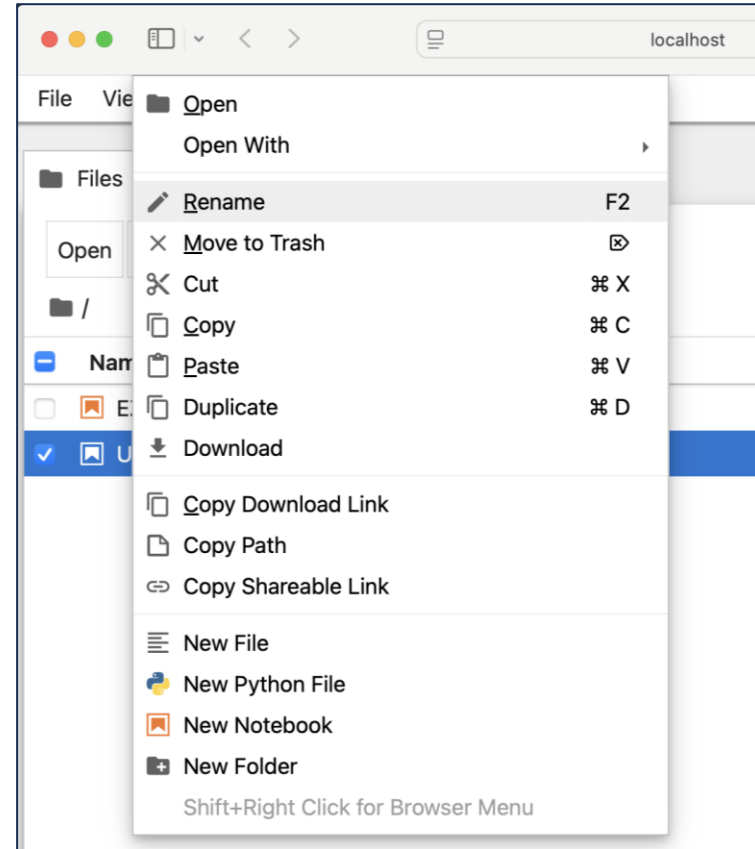
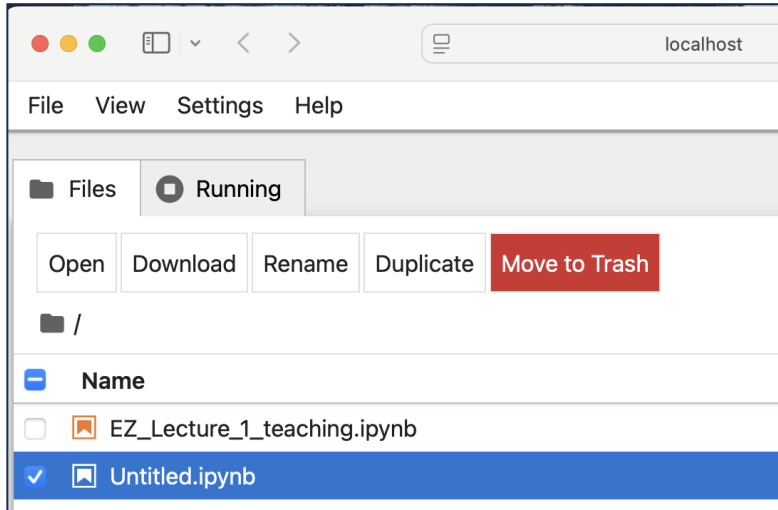
Creating a Notebook

Create a new Notebook:



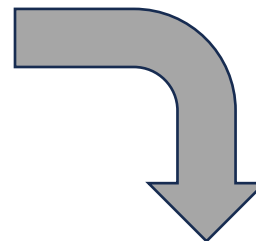
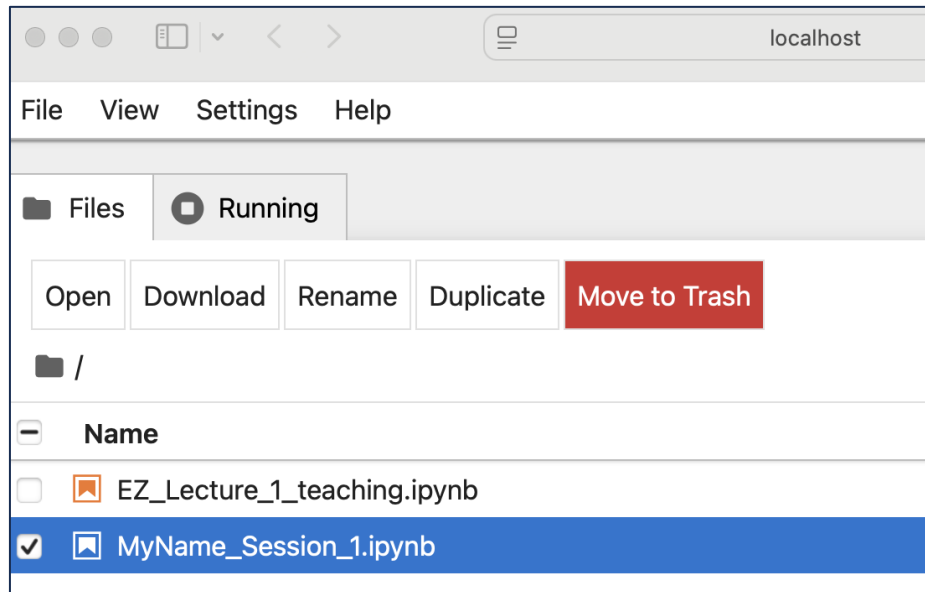
Renaming a Notebook

Rename the file to: YourName_Session_1.ipynb

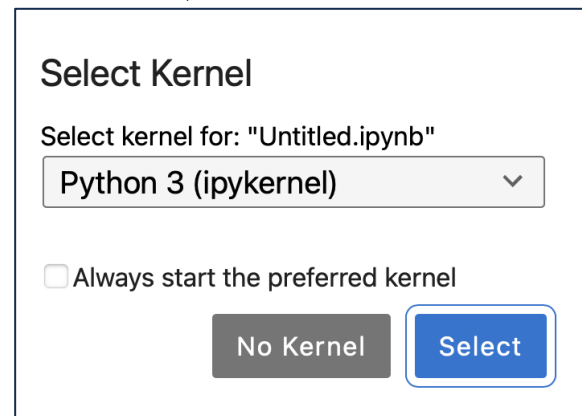


Opening the newly-created Notebook

Open the Notebook that you created (by double-clicking on its icon)

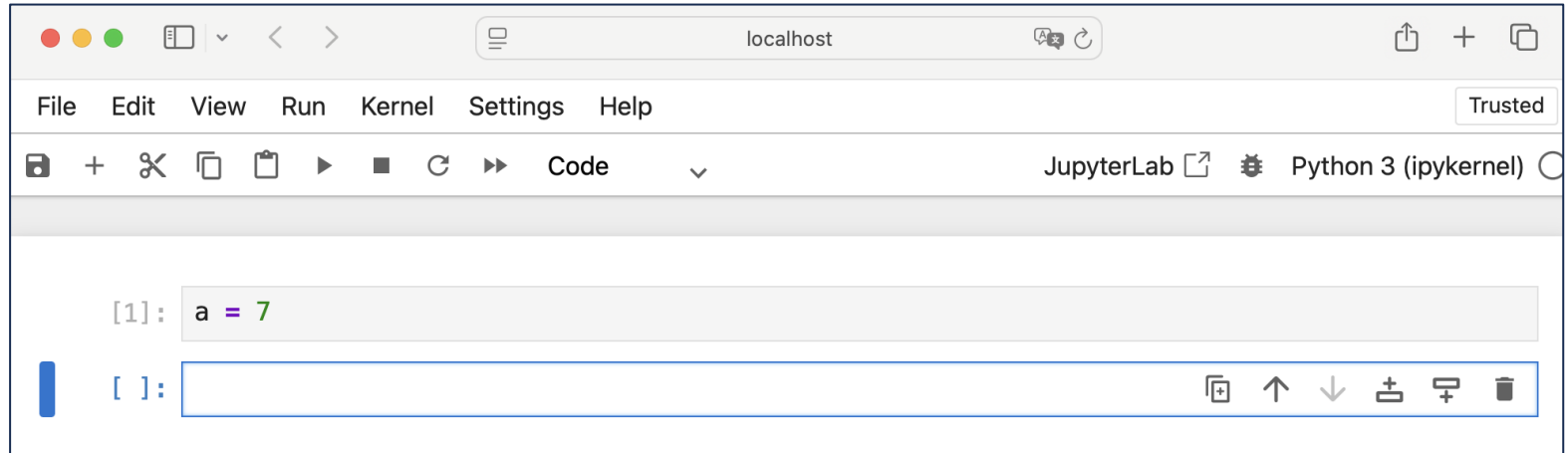
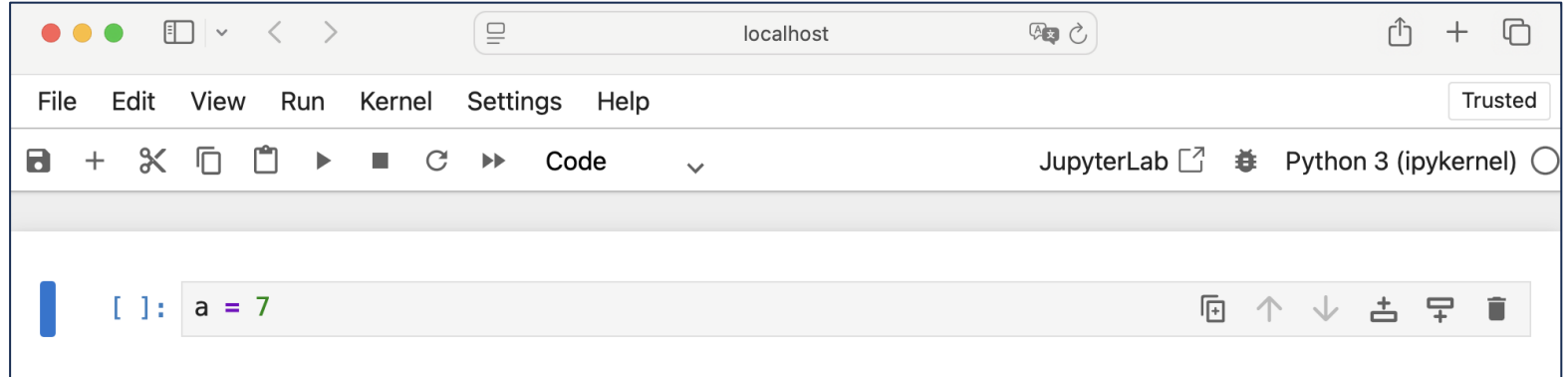


In the poping window
select "Python 3"

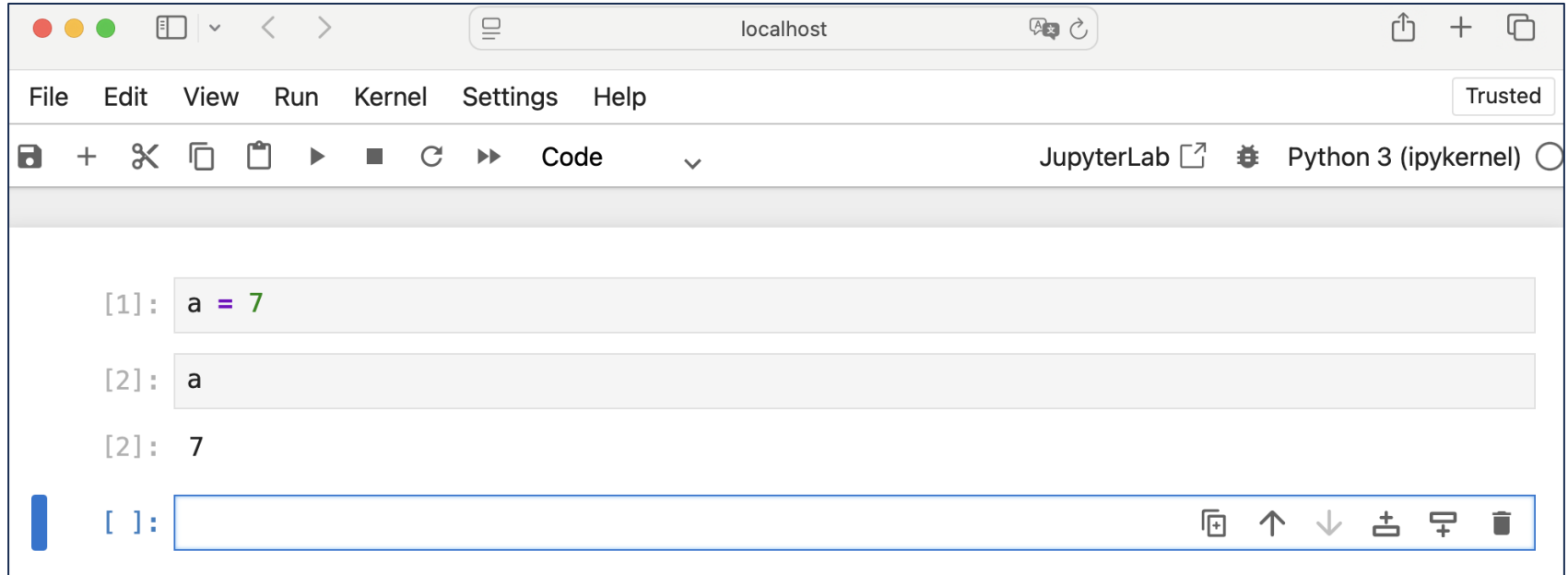


Writing a Python code within a cell and running it

To run a cell: Put the cursor inside the cell, and then press Shift + Enter



Writing a Python code within a cell and running it



Importing a library and using it

Example:

Importing the “Sys” library: `import sys`

Using the imported library, to check the Python version:

```
import sys
```

```
sys.version
```

```
'3.14.3 (v3.14.3:323c59a5e34, Feb  3 2026, 11:41:37) [Clang 16.0.0 (clang-1600.0.26.6)]'
```

Importing libraries

(1) Importing pandas and giving it the alias "pd":

```
import pandas as pd
```

(2) Importing numpy and giving it the alias "np":

```
import numpy as np
```

(3) Importing scipy:

```
import scipy
```

(4) Importing part of matplotlib (pyplot) and giving it the alias "plt":

```
import matplotlib.pyplot as plt
```

(5) Importing seaborn and giving it the alias "sns":

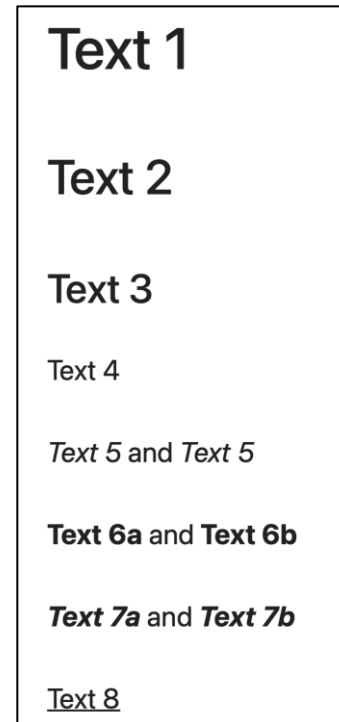
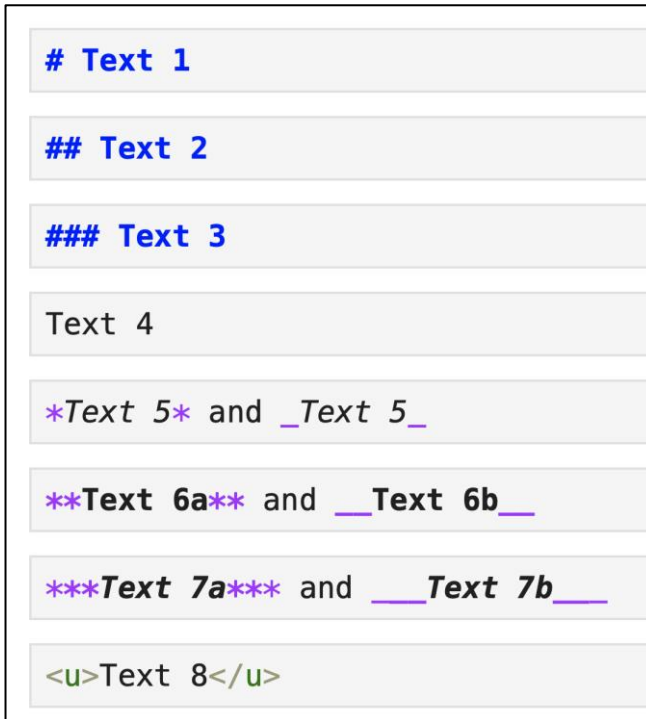
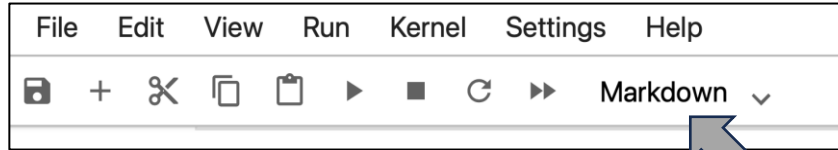
```
import seaborn as sns
```

(6) Importing scikit-learn:

```
import sklearn
```

Jupyter Notebook exercises

Create these cells as Markdown cells, and run them



Getting on the spot documentation for a function

For example, we would like see the documentaton of the function “print”

`print`



`print()`



Put the cursur at the end of the name of the function, or within a () next to it, and press:
Shift + Tab

Signature: `print(*args, sep=' ', end='\n', file=None, flush=False)`

Docstring:

Prints the values to a stream, or to sys.stdout by default.

sep

string inserted between values, default a space.

end

string appended after the last value, default a newline.

file

a file-like object (stream); defaults to the current sys.stdout.

flush

whether to forcibly flush the stream.

Type: builtin_function_or_method

`print`



Getting on the documentation for a function as a cell output

`print?`

Signature: `print(*args, sep=' ', end='\n', file=None, flush=False)`

Docstring:

Prints the values to a stream, or to `sys.stdout` by default.

sep

string inserted between values, default a space.

end

string appended after the last value, default a newline.

file

a file-like object (stream); defaults to the current `sys.stdout`.

flush

whether to forcibly flush the stream.

Type: `builtin_function_or_method`

Exercise

Modify this command:

```
print("a", "b", "c", "d", "e")
```

a b c d e

Without changing the:

```
"a", "b", "c", "d", "e"
```

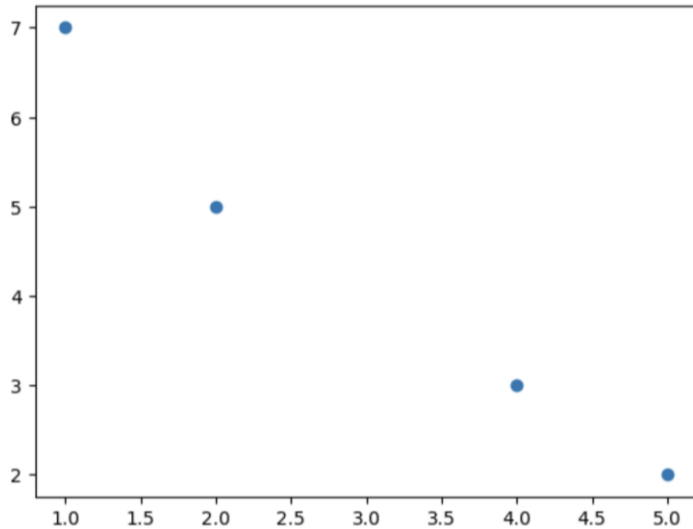
So that the output will be:

(1) a; b; c; d; e

(2) abcde

Exercise: Make a simple plot

```
a = [1, 2, 4, 5]  
b = [7, 5, 3, 2]  
plt.scatter(a,b);
```



Exercise: Change the plot

Modify the `plt.scatter(a,b);` command, to make the size of the markers bigger:

```
a = [1, 2, 4, 5]  
b = [7, 5, 3, 2]  
plt.scatter(a,b);
```

